

AegisFlow

Real-Time Fraud & AML Detection Platform

AegisFlow scores every card and wire transaction in flight, escalates suspicious activity into investigator-ready alerts, and keeps a defensible audit trail for regulators. Milestone 1 builds the streaming backbone, the transactional schema and the pure risk-rule library that every later milestone is layered on.

Milestone 1 Scope

#	M1 Deliverable	What Gets Built
1	Repository scaffold	Public repo, branch policy, ignore rules, agent folder layout
2	Architecture & ADRs	Component map plus two recorded decisions (Kafka over REST, Postgres as system of record)
3	Docker Compose stack	Kafka + Zookeeper, PostgreSQL, Redis, MLflow, Kafka UI — health-checked
4	Database schema	14 tables covering customers, accounts, transactions, alerts, cases, rules and audit
5	Dataset ETL	PaySim / IEEE-CIS download script, DVC-tracked raw zone
6	Domain library	Framework-free rule engine + risk scoring, unit tested
7	Seed data	Reference typologies, sanctions watchlist stub, 5k demo transactions

Infrastructure Stack Provisioned in M1

Service	Image	Purpose
Kafka	confluentinc/cp-kafka:7.6.0	Transaction event bus — topics for raw, scored and alerted events
Zookeeper	confluentinc/cp-zookeeper:7.6.0	Kafka coordination (single-node dev mode)
PostgreSQL	postgres:16-alpine	System of record for accounts, alerts, cases and audit
Redis	redis:7-alpine	Velocity counters and hot feature cache
MLflow	ghcr.io/mlflow/mlflow:v2.14.1	Experiment tracking and model registry from M2 onward
Kafka UI	provectuslabs/kafka-ui:latest	Topic inspection during development

Agent Topology (scaffolded in M1, implemented M2+)

Agent / Service	Responsibility
ingest-gateway	Accepts transaction events over REST, validates the schema, publishes to txn.raw
transaction-scorer	Consumes txn.raw, computes velocity features, applies the rule engine and ML score, publishes to txn.scored
alert-orchestrator	Consumes txn.scored above threshold, deduplicates, opens or updates a case, publishes txn.alerted
case-api	FastAPI service exposing alerts, cases and investigator actions

1.1

Repository Initialisation & Folder Structure

Create the public repo, the M1 skeleton and the ignore rules

```
mkdir aegisflow && cd aegisflow
git init -b main

# --- create the full M1 skeleton in one shot ---
mkdir -p \
  services/ingest_gateway/src \
  services/transaction_scorer/src \
  services/alert_orchestrator/src \
  services/case_api/src \
  libs/aegis_domain \
  config \
  scripts \
  docs/adr \
  data/raw \
  data/interim \
  data/processed \
  tests \
  notebooks

# keep empty data dirs in Git
find data -type d -exec touch {}/.gitkeep \;
```

Resulting tree at the end of M1:

```
aegisflow/
|-- docker-compose.yml
|-- Makefile
|-- .env.example
|-- .gitignore
|-- README.md
|-- config/
|   |-- kafka_topics.yml
|   |-- risk_thresholds.yml
|-- docs/
|   |-- architecture.md
|   |-- adr/
|       |-- 0001-kafka-as-transport.md
|       |-- 0002-postgres-system-of-record.md
|-- libs/
|   |-- aegis_domain/
|       |-- __init__.py
|       |-- rules.py           # pure rule engine
|       |-- scoring.py        # risk band arithmetic
|       |-- models.py         # dataclasses: Transaction, RuleHit, RiskAssessment
|-- services/
|   |-- ingest_gateway/      (Dockerfile, requirements.txt, src/)
|   |-- transaction_scorer/  (Dockerfile, requirements.txt, src/)
|   |-- alert_orchestrator/  (Dockerfile, requirements.txt, src/)
|   |-- case_api/            (Dockerfile, requirements.txt, src/)
|-- scripts/
|   |-- init_db.sql
|   |-- create_topics.sh
|   |-- download_datasets.py
|   |-- seed_db.py
|   |-- verify_m1.py
|-- data/ (raw | interim | processed — DVC tracked)
|-- tests/
|   |-- test_rules.py
```

.gitignore

```
# ---- Python ----
__pycache__/
*.py[cod]
*.egg-info/
.venv/
venv/
.pytest_cache/
.mypy_cache/
.coverage
htmlcov/

# ---- Environment ----
.env
.env.local
*.local.yml

# ---- Editors / OS ----
.vscode/
.idea/
.DS_Store
Thumbs.db

# ---- Model artefacts ----
*.pt
*.pth
*.h5
*.onnx
*.pkl
*.joblib
mlruns/
mlartifacts/

# ---- Data (tracked by DVC, not Git) ----
data/raw/*
data/interim/*
data/processed/*
!data/**/.gitkeep
!data/**/.dvc

# ---- Docker / infra ----
docker-compose.override.yml
*.log
logs/
```

```
# ---- AegisFlow specific ----
data/raw/paysim/
data/raw/ieee-cis/
config/*.secret.yml
kafka-data/
```

```
git commit -m "[M1-AegisFlow] chore: initialise repository skeleton and ignore rules"
```

1.2

README.md — Project Front Door

What a reviewer reads first: problem, stack, quick start

README.md

AegisFlow – Real-Time Fraud & AML Detection Platform

AegisFlow is an agentic fraud and anti-money-laundering platform. Transactions enter over a REST gateway, flow through a Kafka pipeline, are scored by a hybrid rules + ML engine, and surface as investigator-ready cases.

Problem Statement

Card and wire fraud must be judged in under 200 ms, but AML typologies (structuring, layering, rapid movement) only become visible across a window of transactions. A single synchronous model call cannot do both. AegisFlow splits the problem across cooperating agents on a shared event bus: fast deterministic rules inline, heavier behavioural scoring asynchronously, and case management as the system of record.

Architecture (M1 baseline)

```

...
POST /v1/transactions
  |
  v
[ingest-gateway] --publish--> (txn.raw) --> [transaction-scorer]
                                   | Redis velocity counters
                                   | rule engine + ML score
                                   v
                                   (txn.scored)
                                   |
                                   v
                                   [alert-orchestrator]
                                   |           |
                                   | PostgreSQL (txn.alerted)
                                   |           |
                                   v           v
                                   [case-api] --> investigator
...

```

Milestones

Milestone	Focus
M1	Architecture, streaming infrastructure, schema and data foundation
M2	Feature engineering and supervised fraud model training
M3	Real-time scoring service and rule/model blending
M4	Agentic alert triage with an LLM narrative generator
M5	Investigator console and case workflow UI
M6	Deployment, drift monitoring and model governance

```
## Quick Start

```bash
cp .env.example .env
make up # start the full infrastructure stack
make db-init # apply schema (auto-applied on first boot)
make seed # load reference + demo data
make verify # M1 acceptance checks
```

## Repository Layout

- `libs/aegis_domain/` – pure Python rules and scoring, no framework imports
- `services/` – one container per agent
- `scripts/` – schema, topic creation, dataset ETL, seeding, verification
- `config/` – topic definitions and risk thresholds
- `docs/adr/` – architecture decision records

## Tech Stack

- Kafka – Transaction event bus – topics for raw, scored and alerted events
- Zookeeper – Kafka coordination (single-node dev mode)
- PostgreSQL – System of record for accounts, alerts, cases and audit
- Redis – Velocity counters and hot feature cache
- MLflow – Experiment tracking and model registry from M2 onward
- Kafka UI – Topic inspection during development

## License

MIT
```

```
git commit -m "[M1-AegisFlow] docs: project README with architecture and quick start"
```

1.3

Architecture Document & ADRs

Record the design and the two decisions that shape every later milestone

docs/architecture.md

```
# AegisFlow – Architecture

## 1. Context

AegisFlow processes two event classes: card authorisations (high volume, low latency) and wire transfers (low volume, high value). Both land on the same bus so that AML typologies can be evaluated across channels. M1 provisions the bus, the store and the schema; no model is trained yet.

## 2. Component Responsibilities

- ingest-gateway – Accepts transaction events over REST, validates the schema, publishes to txn.raw
- transaction-scorer – Consumes txn.raw, computes velocity features, applies the rule engine and ML score, publishes to txn.scored
- alert-orchestrator – Consumes txn.scored above threshold, deduplicates, opens or updates a case, publishes txn.alerted
- case-api – FastAPI service exposing alerts, cases and investigator actions

## 3. Data Flow

1. `ingest-gateway` validates the payload against a Pydantic schema and publishes to `txn.raw` keyed by `account_id` so all events for an account land on one partition.
2. `transaction-scorer` maintains Redis velocity counters (1m / 1h / 24h windows), evaluates the deterministic rule set, and attaches a risk band.
3. `alert-orchestrator` deduplicates by `(account_id, rule_code, window)`, writes the alert and opens or appends to a case in PostgreSQL.
4. `case-api` serves alerts and cases; every state change writes to `audit_log`.

## 4. Non-Functional Targets

- p95 scoring latency < 200 ms end to end (measured from M3)
- Zero transaction loss: Kafka acks=all, consumer commits after DB write
- Every alert traceable to the exact rule version that fired
- All investigator actions immutable-logged for regulatory review
```

docs/adr/0001-kafka-as-transport.md

```
# ADR-0001: Use Kafka as the inter-agent transport

Status: Accepted
Date: 2026-09-11

## Context

Agents must be independently scalable and must not lose events during a restart. A synchronous REST chain couples availability: if the scorer is down, ingestion fails.

## Decision

All inter-agent communication uses Kafka topics. REST is used only at the edge (`ingest-gateway` inbound, `case-api` outbound). Topics are keyed by `account_id` to guarantee per-account ordering, which AML window rules depend on.

## Consequences

+ Agents restart without data loss; replay is possible for back-testing.
+ Per-account ordering is free.
- Adds Zookeeper and Kafka to the local stack (heavier dev footprint).
- Consumers must be idempotent; alert dedup key is mandatory.
```

docs/adr/0002-postgres-system-of-record.md

```
# ADR-0002: PostgreSQL is the system of record, Redis is disposable

**Status:** Accepted
**Date:** 2026-09-11

## Context

Velocity counters need sub-millisecond reads. Alerts and cases need durability,
referential integrity and auditability for regulators.

## Decision

PostgreSQL holds every durable entity. Redis holds only derived counters with TTLs
and can be flushed at any time without data loss – counters are rebuildable by
replaying Kafka.

## Consequences

+ Clear recovery story: Redis loss is a warm-cache problem, not an outage.
+ Simple audit surface – one database to inspect.
- Counter rebuild after a Redis wipe costs a replay window.
```

```
git commit -m "[M1-AegisFlow] docs: architecture overview and ADR-0001/ADR-0002"
```

1.4

Environment & Service Configuration

Every credential and endpoint the stack needs, with safe defaults

.env.example

```
# ---- PostgreSQL ----
DB_HOST=postgres
DB_PORT=5432
DB_NAME=aegisflow
DB_USER=aegis
DB_PASS=aegis_dev_2026

# ---- Kafka ----
KAFKA_BOOTSTRAP=kafka:29092
TOPIC_RAW=txn.raw
TOPIC_SCORED=txn.scored
TOPIC_ALERTED=txn.alerted
CONSUMER_GROUP_SCORER=aegis-scorer
CONSUMER_GROUP_ORCH=aegis-orchestrator

# ---- Redis ----
REDIS_URL=redis://redis:6379/0
VELOCITY_TTL_SECONDS=86400

# ---- MLflow ----
MLFLOW_TRACKING_URI=http://mlflow:5000

# ---- Risk thresholds (overridable per environment) ----
RISK_REVIEW_THRESHOLD=55
RISK_BLOCK_THRESHOLD=80
STRUCTURING_WINDOW_HOURS=24
STRUCTURING_REPORT_LIMIT=10000
```

config/kafka_topics.yml topic definitions consumed by create_topics.sh

```
topics:
  - name: txn.raw
    partitions: 6
    replication: 1
    config:
      retention.ms: 604800000 # 7 days – enough to replay a week
  - name: txn.scored
    partitions: 6
    replication: 1
    config:
      retention.ms: 604800000
  - name: txn.alerted
    partitions: 3
    replication: 1
    config:
      retention.ms: 2592000000 # 30 days – alerts are reviewed slowly
  - name: txn.dlq
    partitions: 1
    replication: 1
```

scripts/create_topics.sh run once after the stack is healthy

```
#!/usr/bin/env bash
set -euo pipefail
BOOTSTRAP=${KAFKA_BOOTSTRAP:-kafka:29092}

create () {
  docker compose exec -T kafka kafka-topics --bootstrap-server "$BOOTSTRAP" \
    --create --if-not-exists --topic "$1" --partitions "$2" --replication-factor 1
}

create txn.raw 6
create txn.scored 6
create txn.alerted 3
create txn.dlq 1

echo "topics ready:"
docker compose exec -T kafka kafka-topics --bootstrap-server "$BOOTSTRAP" --list
```

config/risk_thresholds.yml weights are versioned; every alert stores the version that fired

```
bands:
  low:      { min: 0,  max: 34, action: allow }
  medium:   { min: 35, max: 54, action: allow_monitor }
  high:     { min: 55, max: 79, action: review }
  critical: { min: 80, max: 100, action: block }

rules:
  R001_HIGH_VALUE:      { weight: 25, threshold_amount: 500000 }
  R002_VELOCITY_1H:     { weight: 20, max_txn_per_hour: 12 }
  R003_STRUCTURING:     { weight: 35, window_hours: 24, report_limit: 1000000 }
  R004_NEW_BENEFICIARY: { weight: 15, account_age_days: 7 }
  R005_HIGH_RISK_COUNTRY: { weight: 30 }
  R006_DORMANT_REACTIVATION: { weight: 20, dormancy_days: 180 }
```

```
git commit -m "[M1-AegisFlow] config: environment template and service configuration files"
```

1.5

Database Schema — Complete DDL

14 tables, indexes and constraints applied on first boot

The schema separates three concerns: **customer and account master data**, the **transaction ledger**, and the **detection outputs** (rule hits, alerts, cases, audit). Detection tables never mutate the ledger — they reference it. This keeps replay safe: re-scoring a week of transactions can only add rows.

| Table | Purpose |
|--------------------|--|
| customers | KYC master record and customer risk rating |
| accounts | Accounts owned by a customer, with open date and dormancy tracking |
| counterparties | Known beneficiaries and their first-seen date per account |
| transactions | Immutable ledger of every card and wire event |
| velocity_snapshots | Point-in-time counter values captured when a transaction was scored |
| rule_definitions | Versioned rule catalogue with weights and parameters |
| rule_hits | Every rule that fired for a transaction, with the version that fired |
| risk_assessments | Final blended score and band per transaction |
| alerts | Deduplicated detections requiring human attention |
| cases | Investigation container grouping related alerts |
| case_alerts | Join table — an alert can be pulled into a case |
| case_notes | Investigator narrative and decision trail |
| watchlist_entries | Sanctions / PEP / internal watchlist stub |
| audit_log | Append-only record of every state change |

scripts/init_db.sql

```
-- =====
-- AegisFlow — M1 schema
-- Applied automatically on first boot via docker-entrypoint-initdb.d
-- =====

CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- =====
-- MASTER DATA
-- =====

CREATE TABLE IF NOT EXISTS customers (
    customer_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    external_ref   VARCHAR(64) UNIQUE NOT NULL,
    full_name      VARCHAR(200) NOT NULL,
    date_of_birth  DATE,
    country_code   CHAR(2) NOT NULL,
    kyc_status     VARCHAR(20) NOT NULL DEFAULT 'pending'
                  CHECK (kyc_status IN ('pending','verified','rejected','expired')),
    risk_rating    VARCHAR(10) NOT NULL DEFAULT 'medium'
                  CHECK (risk_rating IN ('low','medium','high')),
    is_pep         BOOLEAN NOT NULL DEFAULT FALSE,
    onboarded_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS accounts (
    account_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id    UUID NOT NULL REFERENCES customers(customer_id) ON DELETE CASCADE,
    account_number VARCHAR(34) UNIQUE NOT NULL, -- IBAN-width
    account_type   VARCHAR(20) NOT NULL CHECK (account_type IN ('current','savings','card','wallet')),
    currency       CHAR(3) NOT NULL DEFAULT 'INR',
    opened_on      DATE NOT NULL,
    last_active_on DATE,
    status         VARCHAR(20) NOT NULL DEFAULT 'active'
                  CHECK (status IN ('active','dormant','frozen','closed'))
);

CREATE INDEX IF NOT EXISTS idx_accounts_customer ON accounts(customer_id);

CREATE TABLE IF NOT EXISTS counterparties (
    counterparty_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id      UUID NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    beneficiary_ref VARCHAR(64) NOT NULL,
    beneficiary_name VARCHAR(200),
    country_code    CHAR(2),

```

```

    first_seen_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    txn_count        INT NOT NULL DEFAULT 0,
    UNIQUE (account_id, beneficiary_ref)
);

-- =====
-- TRANSACTION LEDGER (immutable)
-- =====

CREATE TABLE IF NOT EXISTS transactions (
    txn_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    external_ref     VARCHAR(64) UNIQUE NOT NULL,
    account_id       UUID NOT NULL REFERENCES accounts(account_id),
    counterparty_id  UUID REFERENCES counterparties(counterparty_id),
    channel          VARCHAR(20) NOT NULL CHECK (channel IN ('card','wire','upi','ach','atm')),
    direction        VARCHAR(6) NOT NULL CHECK (direction IN ('debit','credit')),
    amount           NUMERIC(18,2) NOT NULL CHECK (amount > 0),
    currency         CHAR(3) NOT NULL DEFAULT 'INR',
    merchant_mcc     VARCHAR(4),
    country_code     CHAR(2),
    device_id        VARCHAR(80),
    ip_address       INET,
    occurred_at      TIMESTAMPTZ NOT NULL,
    ingested_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    raw_payload      JSONB
);
CREATE INDEX IF NOT EXISTS idx_txn_account_time ON transactions(account_id, occurred_at DESC);
CREATE INDEX IF NOT EXISTS idx_txn_occurred ON transactions(occurred_at DESC);
CREATE INDEX IF NOT EXISTS idx_txn_amount ON transactions(amount) WHERE amount > 100000;

CREATE TABLE IF NOT EXISTS velocity_snapshots (
    id              BIGSERIAL PRIMARY KEY,
    txn_id          UUID NOT NULL REFERENCES transactions(txn_id) ON DELETE CASCADE,
    txn_count_1m    INT NOT NULL DEFAULT 0,
    txn_count_1h    INT NOT NULL DEFAULT 0,
    txn_count_24h   INT NOT NULL DEFAULT 0,
    amount_sum_24h  NUMERIC(18,2) NOT NULL DEFAULT 0,
    distinct_cp_24h INT NOT NULL DEFAULT 0,
    captured_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_velocity_txn ON velocity_snapshots(txn_id);

-- =====
-- DETECTION

```

```

-- =====

CREATE TABLE IF NOT EXISTS rule_definitions (
  rule_code      VARCHAR(30) NOT NULL,
  version        INT NOT NULL,
  description     TEXT NOT NULL,
  weight         INT NOT NULL CHECK (weight BETWEEN 0 AND 100),
  params         JSONB NOT NULL DEFAULT '{}',
  is_active      BOOLEAN NOT NULL DEFAULT TRUE,
  effective_from  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (rule_code, version)
);

CREATE TABLE IF NOT EXISTS rule_hits (
  id             BIGSERIAL PRIMARY KEY,
  txn_id         UUID NOT NULL REFERENCES transactions(txn_id) ON DELETE CASCADE,
  rule_code      VARCHAR(30) NOT NULL,
  rule_version   INT NOT NULL,
  weight_applied INT NOT NULL,
  evidence       JSONB NOT NULL DEFAULT '{}',
  fired_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  FOREIGN KEY (rule_code, rule_version) REFERENCES rule_definitions(rule_code, version)
);
CREATE INDEX IF NOT EXISTS idx_rule_hits_txn ON rule_hits(txn_id);
CREATE INDEX IF NOT EXISTS idx_rule_hits_code ON rule_hits(rule_code, fired_at DESC);

CREATE TABLE IF NOT EXISTS risk_assessments (
  txn_id         UUID PRIMARY KEY REFERENCES transactions(txn_id) ON DELETE CASCADE,
  rule_score     INT NOT NULL DEFAULT 0,
  model_score    DOUBLE PRECISION,           -- NULL until M2 model exists
  blended_score  INT NOT NULL,
  risk_band     VARCHAR(10) NOT NULL CHECK (risk_band IN ('low','medium','high','critical')),
  decision      VARCHAR(20) NOT NULL CHECK (decision IN ('allow','allow_monitor','review','block')),
  scored_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  scorer_version VARCHAR(40) NOT NULL DEFAULT 'm1-rules-only'
);
CREATE INDEX IF NOT EXISTS idx_risk_band ON risk_assessments(risk_band, scored_at DESC);

-- =====
-- ALERTS, CASES, AUDIT
-- =====

CREATE TABLE IF NOT EXISTS alerts (
  alert_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

    txn_id          UUID NOT NULL REFERENCES transactions(txn_id),
    account_id      UUID NOT NULL REFERENCES accounts(account_id),
    dedup_key       VARCHAR(160) NOT NULL,
    severity        VARCHAR(10) NOT NULL CHECK (severity IN ('low','medium','high','critical')),
    headline        VARCHAR(300) NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'open'
                  CHECK (status IN ('open','triaged','escalated','closed_true','closed_false')),
    raised_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (dedup_key)
);
CREATE INDEX IF NOT EXISTS idx_alerts_status ON alerts(status, raised_at DESC);

CREATE TABLE IF NOT EXISTS cases (
    case_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_number     VARCHAR(20) UNIQUE NOT NULL,
    customer_id     UUID NOT NULL REFERENCES customers(customer_id),
    priority        VARCHAR(10) NOT NULL DEFAULT 'medium',
    status          VARCHAR(20) NOT NULL DEFAULT 'open'
                  CHECK (status IN ('open','investigating','sar_filed','closed')),
    assigned_to     VARCHAR(80),
    opened_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    closed_at       TIMESTAMPTZ
);

CREATE TABLE IF NOT EXISTS case_alerts (
    case_id         UUID NOT NULL REFERENCES cases(case_id) ON DELETE CASCADE,
    alert_id        UUID NOT NULL REFERENCES alerts(alert_id) ON DELETE CASCADE,
    linked_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (case_id, alert_id)
);

CREATE TABLE IF NOT EXISTS case_notes (
    note_id         BIGSERIAL PRIMARY KEY,
    case_id         UUID NOT NULL REFERENCES cases(case_id) ON DELETE CASCADE,
    author          VARCHAR(80) NOT NULL,
    note_type       VARCHAR(20) NOT NULL DEFAULT 'comment',
    body            TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS watchlist_entries (
    entry_id        BIGSERIAL PRIMARY KEY,
    list_name       VARCHAR(40) NOT NULL,      -- 'ofac','un','internal'
    entity_name     VARCHAR(200) NOT NULL,

```

```

    entity_type     VARCHAR(20) NOT NULL DEFAULT 'individual',
    country_code    CHAR(2),
    added_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_watchlist_name ON watchlist_entries(lower(entity_name));

CREATE TABLE IF NOT EXISTS audit_log (
    id              BIGSERIAL PRIMARY KEY,
    ts              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    actor           VARCHAR(80) NOT NULL,
    action          VARCHAR(50) NOT NULL,
    resource_type   VARCHAR(50) NOT NULL,
    resource_id     VARCHAR(100) NOT NULL,
    detail          JSONB NOT NULL DEFAULT '{}'
);
CREATE INDEX IF NOT EXISTS idx_audit_resource ON audit_log(resource_type, resource_id);

```

```
git commit -m "[M1-AegisFlow] config: database schema – 14 tables, indexes and constraints"
```

1.6

Docker Compose — Complete Stack

All infrastructure containers with health checks, volumes and a shared network

`docker-compose.yml`

```
name: aegisflow

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    healthcheck:
      test: ["CMD-SHELL", "echo ruok | nc -w 2 localhost 2181 | grep imok"]
      interval: 10s
      retries: 6
    networks: [aegisnet]

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on:
      zookeeper:
        condition: service_healthy
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:29092,EXTERNAL://localhost:9092
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'false'
    healthcheck:
      test: ["CMD", "kafka-topics", "--bootstrap-server", "kafka:29092", "--list"]
      interval: 15s
      retries: 8
    networks: [aegisnet]

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${DB_USER:-aegis}
      POSTGRES_PASSWORD: ${DB_PASS:-aegis_dev_2026}
      POSTGRES_DB: ${DB_NAME:-aegisflow}
    ports:
      - "5432:5432"
    volumes:
```

```

    - pg_data:/var/lib/postgresql/data
    - ./scripts/init_db.sql:/docker-entrypoint-initdb.d/01_init.sql:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-aegis} -d ${DB_NAME:-aegisflow}"]
    interval: 10s
    retries: 6
  networks: [aegisnet]

redis:
  image: redis:7-alpine
  command: redis-server --appendonly yes --maxmemory 512mb --maxmemory-policy allkeys-lru
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    retries: 5
  networks: [aegisnet]

mlflow:
  image: ghcr.io/mlflow/mlflow:v2.14.1
  command: >
    mlflow server --host 0.0.0.0 --port 5000
    --backend-store-uri sqlite:///mlflow/mlflow.db
    --default-artifact-root /mlflow/artifacts
  ports:
    - "5000:5000"
  volumes:
    - mlflow_data:/mlflow
  networks: [aegisnet]

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  depends_on:
    kafka:
      condition: service_healthy
  ports:
    - "8085:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: aegisflow
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
  networks: [aegisnet]

```

```

volumes:
  pg_data:
  redis_data:
  mlflow_data:

networks:
  aegisnet:
    driver: bridge

```

```
git commit -m "[M1-AegisFlow] config: Docker Compose stack – Kafka, Zookeeper, PostgreSQL, Redis, MLflow, Kafka UI"
```

1.7

Service Dockerfiles & Requirements

One image per agent/service, pinned dependencies

services/ingest_gateway/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

ENV PYTHONUNBUFFERED=1

CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

services/ingest_gateway/requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.30.1
pydantic==2.7.4
confluent-kafka==2.4.0
python-dotenv==1.0.1
```

services/transaction_scorer/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

ENV PYTHONUNBUFFERED=1

CMD ["python", "-m", "src.scorer"]
```

services/transaction_scorer/requirements.txt

```
confluent-kafka==2.4.0
redis==5.0.7
psycpg[binary]==3.1.19
pydantic==2.7.4
pyyaml==6.0.1
python-dotenv==1.0.1
```

services/alert_orchestrator/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

ENV PYTHONUNBUFFERED=1

CMD ["python", "-m", "src.orchestrator"]
```

services/alert_orchestrator/requirements.txt

```
confluent-kafka==2.4.0
psycopg[binary]==3.1.19
pydantic==2.7.4
python-dotenv==1.0.1
```

services/case_api/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

ENV PYTHONUNBUFFERED=1

CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

services/case_api/requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.30.1
psycopg[binary]==3.1.19
pydantic==2.7.4
python-dotenv==1.0.1
```

```
git commit -m "[M1-AegisFlow] feat: service Dockerfiles and pinned requirements"
```

1.8

Makefile — One Command Per Task

The interface every candidate and reviewer uses

Makefile tabs, not spaces, for recipe lines

```
.PHONY: up down logs ps db-init seed verify test lint clean reset

COMPOSE = docker compose -p aegisflow

up:                ## start the full stack
$(COMPOSE) up -d --build
@echo "stack up – run 'make ps' to check health"

down:              ## stop containers, keep volumes
$(COMPOSE) down

reset:             ## stop and wipe all volumes
$(COMPOSE) down -v

ps:                ## container health
$(COMPOSE) ps

logs:              ## tail all logs
$(COMPOSE) logs -f --tail=100

db-init:           ## re-apply schema (idempotent)
docker compose exec -T postgres psql -U ${DB_USER} -d ${DB_NAME} -f
    /docker-entrypoint-initdb.d/01_init.sql

seed:              ## load reference + demo data
python scripts/seed_db.py

data:              ## download raw datasets
python scripts/download_datasets.py

verify:            ## M1 acceptance checks
python scripts/verify_m1.py

test:              ## unit tests for the domain library
pytest tests/ -v

lint:              ## format and lint
black . && flake8 .

clean:
find . -name '__pycache__' -type d -exec rm -rf {} +
```

```
git commit -m "[M1-AegisFlow] chore: Makefile with lifecycle, seed and verification targets"
```

1.9

Dataset Acquisition & DVC Tracking

Raw data pulled reproducibly, versioned outside Git

| Dataset | Source | Used For |
|--------------------------|------------------------------|--|
| PaySim mobile money | Kaggle: ealaxi/paysim1 | Synthetic transfer fraud labels for M2 supervised training |
| IEEE-CIS Fraud Detection | Kaggle: ieee-fraud-detection | Card fraud features and labels |
| OFAC SDN list | treasury.gov sdn.csv | Watchlist screening reference data |

scripts/download_datasets.py

```
#!/usr/bin/env python
"""Download and stage AegisFlow raw datasets into data/raw."""
from __future__ import annotations

import os
import subprocess
import sys
import zipfile
from pathlib import Path
from urllib.request import urlretrieve

RAW = Path(__file__).resolve().parents[1] / 'data' / 'raw'

KAGGLE_SETS = {
    'paysim': 'ealaxi/paysim1',
    'ieee-cis': 'ieee-fraud-detection',
}

DIRECT = {
    'ofac/sdn.csv': 'https://www.treasury.gov/ofac/downloads/sdn.csv',
}

def have_kaggle() -> bool:
    return bool(os.getenv('KAGGLE_USERNAME') and os.getenv('KAGGLE_KEY'))

def fetch_kaggle(name: str, ref: str) -> None:
    target = RAW / name
    if target.exists() and any(target.iterdir()):
        print(f'[skip] {name} already present')
        return
    target.mkdir(parents=True, exist_ok=True)
    cmd = ['kaggle', 'datasets', 'download', '-d', ref, '-p', str(target)]
    if '/' not in ref:
        cmd = ['kaggle', 'competitions', 'download', '-c', ref, '-p', str(target)]
    print(f'[get ] {name} <- {ref}')
    subprocess.run(cmd, check=True)
    for z in target.glob('*.zip'):
        with zipfile.ZipFile(z) as zf:
            zf.extractall(target)
            z.unlink()
```

```
def fetch_direct(rel: str, url: str) -> None:
    target = RAW / rel
    if target.exists():
        print(f'[skip] {rel} already present')
        return
    target.parent.mkdir(parents=True, exist_ok=True)
    print(f'[get ] {rel} <- {url}')
    urlretrieve(url, target)

def main() -> int:
    RAW.mkdir(parents=True, exist_ok=True)
    if have_kaggle():
        for name, ref in KAGGLE_SETS.items():
            fetch_kaggle(name, ref)
    else:
        print('[warn] KAGGLE_USERNAME/KAGGLE_KEY not set – skipping Kaggle datasets.')
        print('      Place the CSVs manually under data/raw/<name>/ and re-run.')
    for rel, url in DIRECT.items():
        try:
            fetch_direct(rel, url)
        except Exception as exc: # network restricted environments
            print(f'[warn] {rel} failed: {exc}')
    print('done. Next: dvc add data/raw && dvc push')
    return 0

if __name__ == '__main__':
    sys.exit(main())
```

DVC initialisation

```
pip install 'dvc[all]'
dvc init
dvc add data/raw
git add data/raw.dvc data/.gitignore .dvc/config

# optional remote (local dir is fine for M1)
dvc remote add -d localstore /tmp/aegisflow-dvcstore
dvc push
```

```
git commit -m "[M1-AegisFlow] data: dataset download script and DVC tracking for data/raw"
```

1.10

Seed Script — Reference & Demo Data

Makes the schema demonstrable the moment the stack boots

scripts/seed_db.py

```
#!/usr/bin/env python
"""Seed AegisFlow reference data and a demo transaction set."""
from __future__ import annotations

import os
import random
import uuid
from datetime import datetime, timedelta, timezone

import psycopg
from psycopg.rows import dict_row

DSN = (
    f"host={os.getenv('DB_HOST', 'localhost')} port={os.getenv('DB_PORT', '5432')} "
    f"dbname={os.getenv('DB_NAME', 'aegisflow')} user={os.getenv('DB_USER', 'aegis')} "
    f"password={os.getenv('DB_PASS', 'aegis_dev_2026')}}"
)

RULES = [
    ('R001_HIGH_VALUE', 1, 'Single transaction above reporting threshold', 25,
     '{"threshold_amount": 500000}'),
    ('R002_VELOCITY_1H', 1, 'More than N transactions in a rolling hour', 20,
     '{"max_txn_per_hour": 12}'),
    ('R003_STRUCTURING', 1, 'Multiple sub-threshold debits summing above the report limit', 35,
     '{"window_hours": 24, "report_limit": 1000000}'),
    ('R004_NEW_BENEFICIARY', 1, 'High-value transfer to a beneficiary first seen recently', 15,
     '{"account_age_days": 7}'),
    ('R005_HIGH_RISK_COUNTRY', 1, 'Counterparty in a high-risk jurisdiction', 30, '{}'),
    ('R006_DORMANT_REACTIVATION', 1, 'Activity on an account dormant beyond N days', 20,
     '{"dormancy_days": 180}'),
]

WATCHLIST = [
    ('internal', 'Vela Holdings Ltd', 'organisation', 'CY'),
    ('internal', 'Arun Bhaskar', 'individual', 'IN'),
    ('ofac', 'Northbridge Trading FZE', 'organisation', 'AE'),
]

HIGH_RISK = ['KP', 'IR', 'SY', 'MM']

def seed_rules(cur) -> None:
    cur.executemany(
        'INSERT INTO rule_definitions (rule_code, version, description, weight, params) '
```

```

        'VALUES (%s, %s, %s, %s, %s::jsonb) ON CONFLICT DO NOTHING',
        RULES,
    )

def seed_watchlist(cur) -> None:
    cur.executemany(
        'INSERT INTO watchlist_entries (list_name, entity_name, entity_type, country_code) '
        'VALUES (%s, %s, %s, %s)',
        WATCHLIST,
    )

def seed_customers(cur, n: int = 120) -> list[str]:
    account_ids: list[str] = []
    for i in range(n):
        cur.execute(
            'INSERT INTO customers (external_ref, full_name, date_of_birth, country_code, '
            'kyc_status, risk_rating, is_pep) VALUES (%s, %s, %s, %s, %s, %s, %s) '
            'RETURNING customer_id',
            (f'CUST{i:05d}', f'Demo Customer {i:03d}', '1988-04-17',
             random.choice(['IN', 'IN', 'IN', 'SG', 'AE']),
             'verified', random.choice(['low', 'low', 'medium', 'high']), i % 37 == 0),
        )
        customer_id = cur.fetchone()['customer_id']
        for a in range(random.randint(1, 2)):
            cur.execute(
                'INSERT INTO accounts (customer_id, account_number, account_type, currency, '
                'opened_on, last_active_on, status) VALUES (%s, %s, %s, %s, %s, %s, %s) '
                'RETURNING account_id',
                (customer_id, f'IN66AEGS{i:07d}{a}', random.choice(['current', 'savings', 'card']),
                 'INR', '2021-06-01', '2026-09-01', 'active'),
            )
            account_ids.append(cur.fetchone()['account_id'])
    return account_ids

def seed_transactions(cur, account_ids: list[str], n: int = 5000) -> None:
    now = datetime.now(timezone.utc)
    rows = []
    for i in range(n):
        account_id = random.choice(account_ids)
        amount = round(random.choice([
            random.uniform(100, 5_000),

```

```

        random.uniform(5_000, 90_000),
        random.uniform(400_000, 900_000),
    ]), 2)
    rows.append((
        f'TXN{i:07d}', account_id,
        random.choice(['card', 'wire', 'upi', 'atm']),
        random.choice(['debit', 'debit', 'credit']),
        amount, 'INR', f'{random.randint(4000, 7999)}',
        random.choice(['IN', 'IN', 'IN', 'SG'] + HIGH_RISK[:1]),
        f'dev-{uuid.uuid4().hex[:10]}',
        now - timedelta(minutes=random.randint(0, 60 * 24 * 30)),
    ))
    cur.executemany(
        'INSERT INTO transactions (external_ref, account_id, channel, direction, amount, '
        'currency, merchant_mcc, country_code, device_id, occurred_at) '
        'VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s) ON CONFLICT DO NOTHING',
        rows,
    )

def main() -> None:
    random.seed(20260911)
    with psycopg.connect(DSN, row_factory=dict_row) as conn:
        with conn.cursor() as cur:
            cur.execute('SELECT COUNT(*) AS c FROM customers')
            if cur.fetchone()['c'] > 0:
                print('already seeded – run `make reset` first to reseed')
                return
            seed_rules(cur)
            seed_watchlist(cur)
            account_ids = seed_customers(cur)
            seed_transactions(cur, account_ids)
            cur.execute(
                "INSERT INTO audit_log (actor, action, resource_type, resource_id, detail) "
                "VALUES ('seed_script', 'SEED', 'database', 'aegisflow', '{\"milestone\": \"M1\"}')"
            )
            conn.commit()
    print('seeded: 6 rules, 3 watchlist entries, 120 customers, ~180 accounts, 5000 transactions')

if __name__ == '__main__':
    main()

```

```
git commit -m "[M1-AegisFlow] data: seed script for reference tables and demo records"
```

1.11

Pure Domain Library

Framework-free business rules — the part that is unit-testable from day one

libs/aegis_domain/rules.py

```

"""Pure rule engine – no database, no Kafka, no framework imports.

Everything here is a deterministic function of the inputs, which is what makes
the rule set unit-testable and replayable.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timedelta
from decimal import Decimal

HIGH_RISK_COUNTRIES = frozenset({'KP', 'IR', 'SY', 'MM', 'AF'})

@dataclass(frozen=True)
class Transaction:
    external_ref: str
    account_id: str
    amount: Decimal
    currency: str
    channel: str
    direction: str
    country_code: str
    occurred_at: datetime
    beneficiary_first_seen_at: datetime | None = None
    account_last_active_on: datetime | None = None

@dataclass(frozen=True)
class Velocity:
    txn_count_1h: int = 0
    txn_count_24h: int = 0
    amount_sum_24h: Decimal = Decimal('0')
    sub_threshold_count_24h: int = 0

@dataclass(frozen=True)
class RuleHit:
    code: str
    version: int
    weight: int
    evidence: dict

```

```

@dataclass
class RuleSet:
    high_value_threshold: Decimal = Decimal('500000')
    max_txn_per_hour: int = 12
    structuring_window_hours: int = 24
    structuring_report_limit: Decimal = Decimal('1000000')
    new_beneficiary_days: int = 7
    dormancy_days: int = 180
    weights: dict = field(default_factory=lambda: {
        'R001_HIGH_VALUE': 25,
        'R002_VELOCITY_1H': 20,
        'R003_STRUCTURING': 35,
        'R004_NEW_BENEFICIARY': 15,
        'R005_HIGH_RISK_COUNTRY': 30,
        'R006_DORMANT_REACTIVATION': 20,
    })

    def evaluate(self, txn: Transaction, vel: Velocity) -> list[RuleHit]:
        hits: list[RuleHit] = []

        if txn.amount >= self.high_value_threshold:
            hits.append(RuleHit('R001_HIGH_VALUE', 1, self.weights['R001_HIGH_VALUE'],
                                {'amount': str(txn.amount),
                                 'threshold': str(self.high_value_threshold)}))

        if vel.txn_count_1h > self.max_txn_per_hour:
            hits.append(RuleHit('R002_VELOCITY_1H', 1, self.weights['R002_VELOCITY_1H'],
                                {'count_1h': vel.txn_count_1h,
                                 'limit': self.max_txn_per_hour}))

        if (vel.sub_threshold_count_24h >= 3
            and vel.amount_sum_24h >= self structuring_report_limit):
            hits.append(RuleHit('R003_STRUCTURING', 1, self.weights['R003_STRUCTURING'],
                                {'sub_threshold_count': vel.sub_threshold_count_24h,
                                 'sum_24h': str(vel.amount_sum_24h)}))

        if txn.beneficiary_first_seen_at is not None:
            age = txn.occurred_at - txn.beneficiary_first_seen_at
            if age < timedelta(days=self.new_beneficiary_days) and txn.amount >= Decimal('100000'):
                hits.append(RuleHit('R004_NEW_BENEFICIARY', 1,
                                    self.weights['R004_NEW_BENEFICIARY'],
                                    {'beneficiary_age_days': age.days}))

        if txn.country_code in HIGH_RISK_COUNTRIES:

```

```

        hits.append(RuleHit('R005_HIGH_RISK_COUNTRY', 1,
                           self.weights['R005_HIGH_RISK_COUNTRY'],
                           {'country': txn.country_code}))

    if txn.account_last_active_on is not None:
        idle = txn.occurred_at - txn.account_last_active_on
        if idle >= timedelta(days=self.dormancy_days):
            hits.append(RuleHit('R006_DORMANT_REACTIVATION', 1,
                               self.weights['R006_DORMANT_REACTIVATION'],
                               {'idle_days': idle.days}))

    return hits

def rule_score(hits: list[RuleHit]) -> int:
    """Capped additive score so a pile-up of small rules cannot exceed 100."""
    return min(100, sum(h.weight for h in hits))

def risk_band(score: int) -> str:
    if score >= 80:
        return 'critical'
    if score >= 55:
        return 'high'
    if score >= 35:
        return 'medium'
    return 'low'

def decision_for(band: str) -> str:
    return {'low': 'allow', 'medium': 'allow_monitor',
            'high': 'review', 'critical': 'block'}[band]

def dedup_key(account_id: str, codes: list[str], occurred_at: datetime) -> str:
    """Stable key so replays do not create duplicate alerts."""
    bucket = occurred_at.strftime('%Y%m%d%H')
    return f"{account_id}|{'+'.join(sorted(codes))}|{bucket}"

```

tests/test_rules.py

```

from datetime import datetime, timedelta, timezone
from decimal import Decimal

import pytest

from libs.aegis_domain.rules import (
    RuleSet, Transaction, Velocity, dedup_key, decision_for, risk_band, rule_score,
)

NOW = datetime(2026, 9, 11, 10, 30, tzinfo=timezone.utc)

def txn(**over):
    base = dict(external_ref='T1', account_id='A1', amount=Decimal('1000'),
                currency='INR', channel='wire', direction='debit',
                country_code='IN', occurred_at=NOW)
    base.update(over)
    return Transaction(**base)

def test_clean_transaction_produces_no_hits():
    assert RuleSet().evaluate(txn(), Velocity()) == []

def test_high_value_fires_at_threshold():
    hits = RuleSet().evaluate(txn(amount=Decimal('500000')), Velocity())
    assert [h.code for h in hits] == ['R001_HIGH_VALUE']

def test_velocity_fires_above_limit_only():
    rs = RuleSet()
    assert rs.evaluate(txn(), Velocity(txn_count_1h=12)) == []
    assert rs.evaluate(txn(), Velocity(txn_count_1h=13))[0].code == 'R002_VELOCITY_1H'

def test_structuring_needs_both_count_and_sum():
    rs = RuleSet()
    only_sum = Velocity(sub_threshold_count_24h=1, amount_sum_24h=Decimal('2000000'))
    assert rs.evaluate(txn(), only_sum) == []
    both = Velocity(sub_threshold_count_24h=4, amount_sum_24h=Decimal('2000000'))
    assert rs.evaluate(txn(), both)[0].code == 'R003_STRUCTURING'

def test_new_beneficiary_requires_material_amount():

```

```

rs = RuleSet()
recent = NOW - timedelta(days=2)
small = txn(amount=Decimal('5000'), beneficiary_first_seen_at=recent)
assert rs.evaluate(small, Velocity()) == []
large = txn(amount=Decimal('250000'), beneficiary_first_seen_at=recent)
assert 'R004_NEW_BENEFICIARY' in [h.code for h in rs.evaluate(large, Velocity())]

def test_dormant_reactivation():
    rs = RuleSet()
    t = txn(account_last_active_on=NOW - timedelta(days=200))
    assert 'R006_DORMANT_REACTIVATION' in [h.code for h in rs.evaluate(t, Velocity())]

@pytest.mark.parametrize('score,band', [(0, 'low'), (34, 'low'), (35, 'medium'),
                                          (55, 'high'), (80, 'critical'), (100, 'critical')])
def test_band_boundaries(score, band):
    assert risk_band(score) == band

def test_score_is_capped_at_100():
    t = txn(amount=Decimal('900000'), country_code='KP',
            beneficiary_first_seen_at=NOW - timedelta(days=1),
            account_last_active_on=NOW - timedelta(days=400))
    v = Velocity(txn_count_1h=40, sub_threshold_count_24h=9,
                amount_sum_24h=Decimal('5000000'))
    assert rule_score(RuleSet().evaluate(t, v)) == 100

def test_decision_mapping():
    assert decision_for('critical') == 'block'
    assert decision_for('low') == 'allow'

def test_dedup_key_is_order_independent_and_hour_bucketed():
    a = dedup_key('A1', ['R002_VELOCITY_1H', 'R001_HIGH_VALUE'], NOW)
    b = dedup_key('A1', ['R001_HIGH_VALUE', 'R002_VELOCITY_1H'], NOW + timedelta(minutes=20))
    assert a == b

```

```
git commit -m "[M1-AegisFlow] feat: pure domain library with unit tests"
```

1.12

M1 Verification & Milestone Tag

Prove every deliverable before tagging v0.1.0-M1

| Acceptance Check | Command | Expected |
|------------------------|--|---|
| All containers healthy | <code>make ps</code> | 6 services, status healthy |
| Schema applied | <code>psql -c "\dt"</code> | 14 tables present |
| Kafka topics created | <code>bash scripts/create_topics.sh</code> | txn.raw, txn.scored, txn.alerted, txn.dlq |
| Rule catalogue seeded | <code>SELECT count(*) FROM rule_definitions</code> | 6 rows |
| Demo ledger seeded | <code>SELECT count(*) FROM transactions</code> | 5000 rows |
| Domain tests green | <code>pytest tests/ -v</code> | 10 passed |
| Raw data tracked | <code>dvc status</code> | data/raw.dvc up to date |

scripts/verify_m1.py

```
#!/usr/bin/env python
"""M1 acceptance checks for AegisFlow. Exit code 0 means the milestone is done."""
from __future__ import annotations

import os
import subprocess
import sys

import psycpg

DSN = (
    f"host={os.getenv('DB_HOST', 'localhost')} port={os.getenv('DB_PORT', '5432')} "
    f"dbname={os.getenv('DB_NAME', 'aegisflow')} user={os.getenv('DB_USER', 'aegis')} "
    f"password={os.getenv('DB_PASS', 'aegis_dev_2026')}"
)

EXPECTED_TABLES = {
    'customers', 'accounts', 'counterparties', 'transactions', 'velocity_snapshots',
    'rule_definitions', 'rule_hits', 'risk_assessments', 'alerts', 'cases',
    'case_alerts', 'case_notes', 'watchlist_entries', 'audit_log',
}

failures: list[str] = []

def check(label: str, ok: bool, detail: str = '') -> None:
    print(f"{'PASS' if ok else 'FAIL'} {label}{' - ' + detail if detail else ''}")
    if not ok:
        failures.append(label)

def main() -> int:
    with psycpg.connect(DSN) as conn, conn.cursor() as cur:
        cur.execute("SELECT table_name FROM information_schema.tables "
                    "WHERE table_schema = 'public'")
        found = {r[0] for r in cur.fetchall()}
        missing = EXPECTED_TABLES - found
        check('schema: all 14 tables present', not missing,
              f'missing {sorted(missing)}' if missing else '14/14')

        cur.execute('SELECT count(*) FROM rule_definitions')
        n = cur.fetchone()[0]
        check('seed: rule catalogue', n >= 6, f'{n} rules')

    return 0 if not failures else 1
```

```

cur.execute('SELECT count(*) FROM transactions')
n = cur.fetchone()[0]
check('seed: demo ledger', n >= 1000, f'{n} transactions')

cur.execute('SELECT count(*) FROM watchlist_entries')
n = cur.fetchone()[0]
check('seed: watchlist', n >= 3, f'{n} entries')

rc = subprocess.run(['pytest', 'tests/', '-q'], capture_output=True, text=True)
check('domain: unit tests', rc.returncode == 0, rc.stdout.strip().splitlines()[-1]
      if rc.stdout.strip() else '')

print()
if failures:
    print(f'M1 NOT COMPLETE - {len(failures)} check(s) failed')
    return 1
print('M1 COMPLETE - safe to tag v0.1.0-M1')
return 0

if __name__ == '__main__':
    sys.exit(main())

```

Tag the milestone

Only tag once make verify is green. The tag is what the reviewer checks out.

```

make up && make seed && make verify

git add -A
git commit -m "[M1-AegisFlow] chore: M1 verification script"
git tag -a v0.1.0-M1 -m "AegisFlow Milestone 1 - architecture, infrastructure, schema, data foundation"
git push origin main --tags

```

M1 Definition of Done

- Public GitHub repo with the M1 folder structure and a protected main branch.
- `docker compose up` brings six services to healthy with no manual steps.
- All 14 tables created automatically on first boot; schema is idempotent and re-runnable.
- Four Kafka topics exist with the documented partition counts and retention.
- Rule catalogue, watchlist and a 5,000-row demo ledger are seeded and queryable.
- The pure domain library passes its full unit-test suite with no infrastructure running.
- Raw datasets are DVC-tracked; only the `.dvc` pointers are in Git.
- Architecture document and two ADRs are committed and reviewed.
- Repository tagged `v0.1.0-M1`.

What M2 picks up from here

M2 trains the supervised fraud model on the PaySim and IEEE-CIS data staged here. The velocity tables and rule catalogue become the feature store contract, and `risk_assessments.model_score` — deliberately nullable in M1 — is where the trained model writes its output.